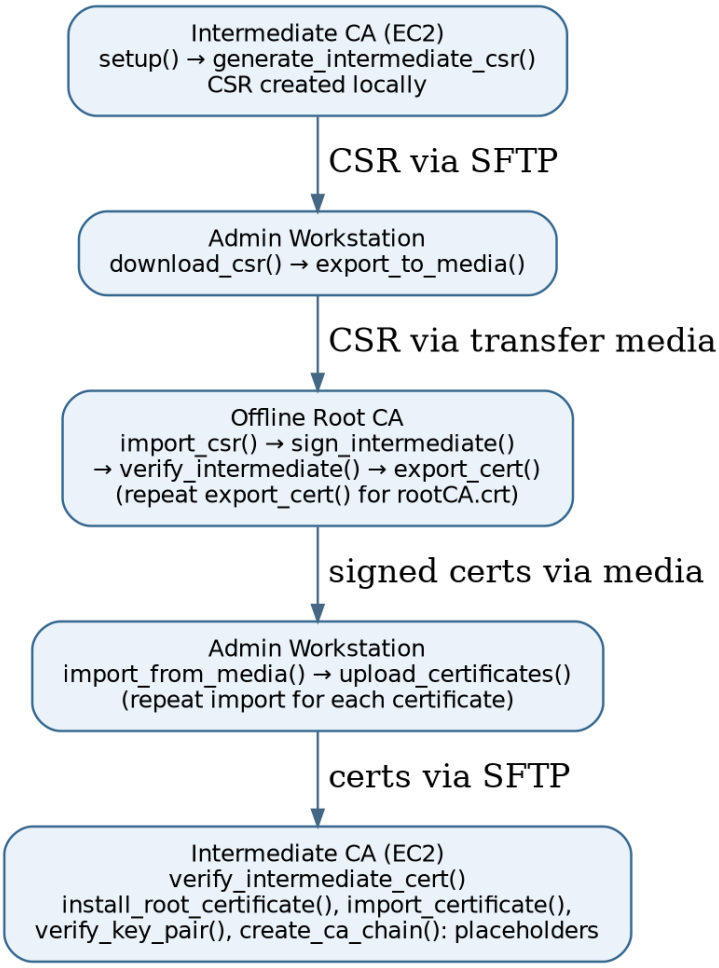
# PKI Operational Workflows

Verified function-call sequences from the provided Root CA, Admin Workstation, and Intermediate CA Python source files. Updated 20 October 2026.

## 1. Intermediate CA CSR signing round trip




**1. Intermediate CA (EC2):** intermediate_setup.setup(mode="classical") creates CA directories, keys, OpenSSL configuration and CSR through generate_intermediate_csr(ca_dir, ca_name). Run only as needed; setup() currently initializes both intermediate CAs.

**2. Admin Workstation:** download_csr(hostname, username, key_path, remote_csr_path, local_dir) retrieves CSR over SFTP; export_to_media(source_path, transfer_dir) stages it on transfer media.

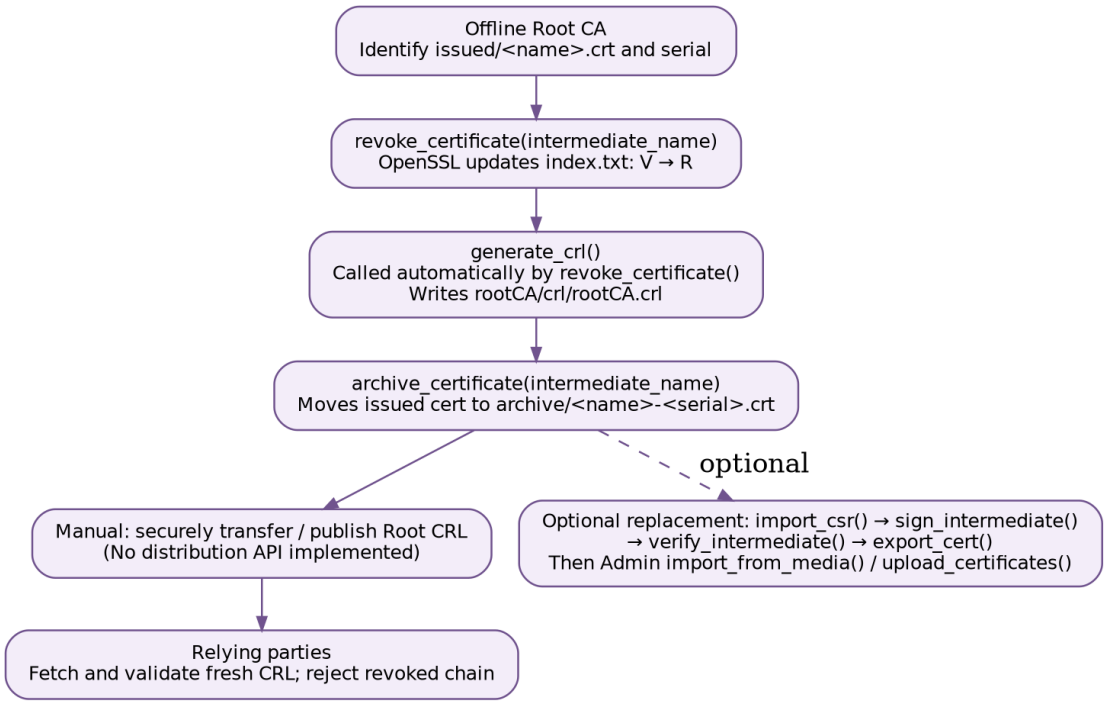
**3. Offline Root CA:** import_csr(csr_name, source_dir=..., dest_dir=...) verifies CSR signature and imports it. Separate CA authorization/approval is a manual policy step, not an implemented function.

**4. Offline Root CA:** sign_intermediate(intermediate_name, source_dir=..., dest_dir=...) issues the certificate; verify_intermediate(intermediate_cert_file, ...) checks its chain. export_cert(cert_name, source_dir=None, transfer_dir=...) exports the intermediate certificate, and a second call with source_dir=root_dir / "certs" exports rootCA.crt.

**5. Admin Workstation:** import_from_media(cert_name, transfer_dir, destination_dir, overwrite=False) copies each returned certificate to staging; upload_certificates(hostname, username, key_path, local_cert_dir, remote_cert_dir, cert_names) uploads one or more certificates to EC2.

**6. Intermediate CA (EC2):** verify_intermediate_cert(ca_dir, ca_name) checks the intermediate certificate against the root certificate after both files are present. import_certificate(), install_root_certificate(), verify_key_pair(), and create_ca_chain() are declared but currently contain pass; certificate placement is handled by the Admin upload.

## 2. Intermediate CA certificate revocation




**1. Root CA:** Identify the correct issued intermediate CA certificate and its serial before revocation.

**2. Root CA:** revoke_certificate(intermediate_name, cert_dir=...) calls OpenSSL ca -revoke with reason superseded and then automatically calls generate_crl().

**3. Root CA:** generate_crl(crl_dir=...) creates rootCA/crl/rootCA.crl; inspect with openssl crl -in rootCA/crl/rootCA.crl -noout -text.

**4. Root CA:** archive_certificate(intermediate_name, source_dir=..., archive_dir=...) moves the revoked certificate to a serial-numbered archive path.

**5. Manual distribution:** Transfer and publish the signed Root CRL to a reachable location. No CRL publishing API is implemented in the shared code.

**6. Optional replacement:** Generate a fresh intermediate CSR and repeat the signing round trip. Existing file protections may require deliberate cleanup/archiving before import or upload.

## Implementation boundaries

- Revocation and CRL generation are implemented by the offline Root CA; CRL publication and relying-party checks are external/manual.

- The Root CA import_csr() validates the CSR signature but does not implement an authorization or approval workflow.
- Intermediate CA operational methods with pass are placeholders, not executable steps.
- The Admin Workstation coordinates transfers; it does not sign or revoke certificates.
- CRL generation is already invoked by revoke_certificate(); an extra call is optional and generates a newer CRL number.